Python Functions

What is a Function?

- A **function** is a block of code that runs only when it is called.
- You can pass **data (parameters)** into a function.
- A function can **return data (output)** back to the caller.
- Functions in Python are defined using the def keyword.

Syntax

```
def function_name(parameters):
    # function body
    return value
```

- **Parameter**: variable in function definition.
- **Argument**: actual value passed when calling the function.

Function Examples

1. No parameters, no return value

```
def myfun():
    print("Hello")

myfun()   # Output: Hello
```

2. Function with parameters, no return value

```
def myfun(name):
    print("Hello", name)

myfun("Smith")   # Output: Hello Smith
```

3. Function with parameters and return value

```
def cal(a, b):
    return a + b

print(cal(10, 20))   # Output: 30
```

4. Function returning None

```
def myfunc():
    return

print(myfunc())   # Output: None
```

Types of Function Arguments

1. **Arbitrary (Variable-length) Arguments** – use *args
2. **Positional (Required) Arguments**
3. **Keyword Arguments**

Example 1: Arbitrary Arguments (*args)

If you don't know how many arguments will be passed, use * before the parameter name.
It will be received as a **tuple**.

```
def sum_function(*numbers):
    total = 0
    for i in numbers:
        total += i
    return total

print(sum_function(1, 2))      # 3
print(sum_function(1, 2, 3))   # 6
print(sum_function())          # 0
```

Example 2: Positional & Keyword Arguments

```
def myfunc(i, j):
    print(i, j)

myfunc(10, 20)        # Positional arguments → 10 20
```

```
myfunc(j=20, i=10)     # Keyword arguments → 10 20
```

Example 3: Default Arguments

You can assign default values to parameters.

```
def myfunc(i, j=10):
    print(i, j)

myfunc(100, 200)   # 100 200
myfunc(100)        # 100 10
```

Example 4: Keyword Arguments

```
def greetings(name, greetmsg):
    print(greetmsg + " " + name)

greetings(name='John', greetmsg="Hello")   # Hello John
greetings(greetmsg="Hi", name='Alice')     # Hi Alice
```

Example 5: Mixing Positional & Keyword Arguments

```
def myfunc(a, b, c):
    print(a, b, c)

myfunc(10, 20, 30)            # Positional
myfunc(a=10, b=20, c=30)      # All Keyword
myfunc(10, 20, c=30)          # Mix → 10 20 30
```

⚠️ **Rule**: Positional arguments must come **before** keyword arguments.
❌ myfunc(10, b=20, 30) → Error

Example 6: Function Returning Multiple Values

```
def largest(a, b):
    if a > b:
        return a, b
    else:
        return b, a

res = largest(10, 20)
print(res)        # (20, 10)
print(type(res))    # tuple
```

Scope of Variables in Functions

- **Global Variable**: Declared outside functions, accessible everywhere.
- **Local Variable**: Declared inside a function, accessible only inside it.

Example 1: Global & Local Variables

```
x = 20   # Global

def myfunc():
    y = 10  # Local
    print(x)   # Access global
    print(y)

myfunc()
print(x)   # OK
print(y)   # Error → local variable not accessible
```

Example 2: Same Name for Global & Local

```
x = 100   # Global

def myfunc():
    x = 200   # Local
    print(x)  # Local variable used

myfunc()   # Output: 200
```

Example 3: Updating Global Variable Inside Function

```
x = 100

def myfunc():
    global x
    x = 200   # Modifies global variable
    print(x)

myfunc()    # 200
print(x)    # 200
```

Example 4: Declaring Global Variable Inside Function

```
def myfunc():
    global x
    x = 100
    print(x)

myfunc()    # 100
print(x)    # 100 (accessible globally)
```

Key Takeaways

- Use ***def*** to define functions.
- Parameters = placeholders, Arguments = actual values.
- Functions can return single or multiple values.
- Global variables are accessible everywhere, but **prefer local variables** inside functions for better practice.